

The Full-Stack Architecture

Backend & AI (The Engine Room)

- **Docker:** Essential for containerizing Gorse, Redis, and Typesense so they run consistently on any machine.
- **Gin:** A high-performance HTTP web framework for **Go**. This will be your "API Gateway" that talks to the frontend.
- **Go (Golang):** The primary language for your backend services, chosen for its speed and concurrency.
- **Gorse:** The open-source recommendation engine. It handles the **Collaborative Filtering** and **AutoML** (automatically finding the best model for your data).
- **Redis:** Used by Gorse for **caching** intermediate results and session data to ensure the feed loads in milliseconds.
- **Typesense:** Your **Vector Search** database. It handles "Candidate Generation" by finding posts with similar embeddings (e.g., "Show me posts similar to this video about coding").

Frontend & Mobile (The User Experience)

- **Lucide React:** A clean icon library that integrates perfectly with Shadcn.
- **Next.js (v14/15):** The React framework for your web platform, utilizing **Server Components** for SEO and speed.
- **React Native:** For the mobile app, allowing you to share logic with your Next.js project via a "Monorepo" setup.
- **Redux Toolkit (RTK):** The industry standard for managing global state (e.g., "Is the user logged in?").
- **RTK Query:** The "secret sauce" for data fetching. It handles caching, loading states, and optimistic updates (making a "Like" feel instant).
- **Shadcn UI:** A collection of re-usable components built on Radix UI and Tailwind CSS. It's what makes modern apps look "clean."
- **Tailwind CSS:** For rapid, utility-first styling across both web and mobile.
- **Zod:** For schema validation, ensuring the data coming from your Go API is exactly what your frontend expects.

How They Work Together (The Flow)

1. **Ingestion:** A user likes a post in your **React Native** app.
2. **Signal:** **RTK Query** sends that "Like" to your **Gin (Go)** backend.
3. **Storage:** The backend saves the interaction into **Gorse**.
4. **Training:** **Gorse** updates the user's profile in the background using **Redis**.
5. **Retrieval:** When the user refreshes their feed, **Gorse** asks **Typesense** for similar content based on vectors.
6. **Display:** The ranked list of posts is sent back to **Next.js**, which renders them using **Shadcn** components.

Pro-Tip for your Graduation Defense

When the professors ask why you used this specific stack, give them this "**Industry Alignment**" answer:

*"I chose **Go** and **Gorse** for the backend to match the high-concurrency requirements of real-time social media, similar to X's use of Scala. I used **Typesense** for vector search because it provides sub-50ms latency for candidate generation, and **RTK Query** on the frontend to ensure a seamless, 'optimistic' user experience where interactions feel instantaneous."*